

PLAYWRIGHT

TypeScript

Complete Study Material

-
- ✓ Utility Functions & Import/Export Modules
 - ✓ Configuration File & Cross-Browser Execution
 - ✓ Parallel Execution, Retries & Workers
 - ✓ Test Suite & Test Level Configurations
 - ✓ Screenshots, Tracing & Debugging
 - ✓ Playwright Hooks: before / after / beforeAll / afterAll
 - ✓ Framework Architecture Explained

Comprehensive Guide with Code Examples & Best Practices

Table of Contents

1. Introduction to Utilities

- What are Utilities in Playwright?
- Import & Export Modules (ESM / CommonJS)
- Creating Reusable Utility Functions

2. Configuration File

- playwright.config.ts Deep Dive
- Cross-Browser Execution
- Running Tests in Parallel
- Retries & Worker Configuration

3. Test Suite & Test Level Config

- Structuring Test Suites
- Test-Level Configuration Overrides
- Attaching Screenshots to HTML Report

4. Tracing & Debugging

- Enabling & Viewing Traces
- Playwright Inspector
- Debug Flags & Console Logging

5. Playwright Hooks

- beforeAll & afterAll
- beforeEach & afterEach
- Practical Hook Script Examples

6. Framework Explanation

- Playwright Architecture Overview
- Page Object Model (POM)
- Project Structure Best Practices

CHAPTER 01

Introduction to Utilities

Import-Export Modules & Creating Utility Functions

What Are Utilities in Playwright?

Utilities are reusable helper functions and modules that eliminate code duplication across your test suite. In Playwright with TypeScript, utilities typically handle common tasks such as login flows, data generation, API calls, date formatting, and custom assertions. Organizing code into utility modules improves maintainability, readability, and test reliability.

- Centralise repetitive logic (e.g., login, API helpers)
- Type-safe through TypeScript interfaces and generics
- Shared across multiple spec files via import/export
- Easier to unit-test in isolation

Import & Export Modules

TypeScript supports both ES Modules (ESM) and CommonJS. Playwright projects configured with **esm: true** in tsconfig use the **import/export** syntax. Below are the key patterns:

Named Exports

```
// utils/helpers.ts
import { Page } from '@playwright/test';

// Named export - can export multiple from one file
export async function loginUser(page: Page, username: string, password: string) {
  await page.goto('/login');
  await page.fill('#username', username);
  await page.fill('#password', password);
  await page.click('button[type="submit"]');
  await page.waitForURL('/dashboard');
}

export function generateEmail(prefix: string = 'user'): string {
  return `${prefix}_${Date.now()}@test.com`;
}
```

Default Export

```
// utils/ApiClient.ts
export default class ApiClient {
  private baseUrl: string;

  constructor(baseUrl: string) {
    this.baseUrl = baseUrl;
  }

  async getUser(id: number) {
    const res = await fetch(`${this.baseUrl}/users/${id}`);
    return res.json();
  }
}
```

Importing in Spec Files

```
// tests/login.spec.ts
import { test, expect } from '@playwright/test';
import { loginUser, generateEmail } from '../utils/helpers';
import ApiClient from '../utils/ApiClient';

test('User can login', async ({ page }) => {
  const email = generateEmail('admin');
  await loginUser(page, email, 'SecurePass123');
  await expect(page).toHaveURL('/dashboard');
});
```

■ *Note: Always use relative paths starting with `../` or `./` when importing local modules. Configure path aliases in `tsconfig.json` for cleaner imports like `@utils/helpers`.*

Barrel Files (index.ts)

A barrel file re-exports everything from a directory, giving consumers a single import path:

```
// utils/index.ts (barrel file)
export { loginUser, generateEmail } from './helpers';
export { default as ApiClient } from './ApiClient';
export * from './dateUtils';

// Consumer can now write:
import { loginUser, ApiClient } from '../utils';
```

Creating Utility Functions

Utility functions should be pure, focused, and strongly typed. Common categories:

1. Page Interaction Helpers

```
// utils/pageUtils.ts
import { Page, Locator } from '@playwright/test';

export async function waitAndClick(page: Page, selector: string, timeout = 5000) {
  const el = page.locator(selector);
  await el.waitFor({ state: 'visible', timeout });
  await el.click();
}

export async function fillForm(page: Page, data: Record<string, string>) {
  for (const [field, value] of Object.entries(data)) {
    await page.fill(`[name='${field}']`, value);
  }
}

export async function getTableData(page: Page, tableSelector: string) {
  const rows = page.locator(`${tableSelector} tbody tr`);
  const count = await rows.count();
  const data: string[][] = [];
  for (let i = 0; i < count; i++) {
    const cells = await rows.nth(i).locator('td').allTextContents();
    data.push(cells);
  }
  return data;
}
```

2. Data Generation Helpers

```
// utils/dataUtils.ts
export interface UserData {
  name: string;
  email: string;
  phone: string;
}

export function generateUser(overrides?: Partial<UserData>): UserData {
  const ts = Date.now();
  return {
    name: `Test User ${ts}`,
    email: `user${ts}@example.com`,
    phone: '9876543210',
    ...overrides,
  };
}

export function randomInt(min: number, max: number): number {
  return Math.floor(Math.random() * (max - min + 1)) + min;
}
```

■ Tip: Type your utility return values explicitly. It helps IDEs autocomplete and catches bugs at compile time, not at runtime.

CHAPTER 02

Configuration File

Cross-Browser Execution • Parallel Tests • Retries & Workers

playwright.config.ts – Overview

The **playwright.config.ts** file is the central control panel for your test suite. It defines browsers, base URLs, timeouts, retries, parallelism, reporters, and much more. Playwright auto-discovers this file at the project root.

Full Configuration Example

```
// playwright.config.ts
import { defineConfig, devices } from '@playwright/test';

export default defineConfig({
  // Directory containing test files
  testDir: './tests',

  // Run all tests in parallel
  fullyParallel: true,

  // Fail the build on CI if you accidentally left test.only
  forbidOnly: !!process.env.CI,

  // Retry on failure (2 retries on CI, 0 locally)
  retries: process.env.CI ? 2 : 0,

  // Number of worker threads
  workers: process.env.CI ? 4 : undefined,

  // Reporter configuration
  reporter: [
    ['html', { outputFolder: 'playwright-report', open: 'never' }],
    ['list'],
  ],

  // Shared settings for all projects
  use: {
    baseURL: 'http://localhost:3000',
    trace: 'on-first-retry',
    screenshot: 'only-on-failure',
    video: 'retain-on-failure',
    headless: true,
    actionTimeout: 10_000,
    navigationTimeout: 30_000,
  },

  // Cross-browser projects
  projects: [
    { name: 'chromium', use: { ...devices['Desktop Chrome'] } },
    { name: 'firefox', use: { ...devices['Desktop Firefox'] } },
    { name: 'webkit', use: { ...devices['Desktop Safari'] } },
    { name: 'mobile-chrome', use: { ...devices['Pixel 5'] } },
  ],
}
```



```
{ name: 'mobile-safari', use: { ...devices['iPhone 12'] } },
],
});
```

Cross-Browser Execution

Playwright supports Chromium, Firefox, and WebKit natively. Each **project** in the config represents a browser + device combination. All your tests run against every project by default.

Project Name	Browser Engine	Represents
chromium	Chromium	Google Chrome, Microsoft Edge
firefox	Firefox	Mozilla Firefox
webkit	WebKit	Apple Safari
mobile-chrome	Chromium	Android Chrome (emulated)
mobile-safari	WebKit	iOS Safari (emulated)

Running specific browsers from CLI:

```
# Run only Chromium tests
npx playwright test --project=chromium

# Run Firefox and WebKit
npx playwright test --project=firefox --project=webkit

# Run a specific test file on all browsers
npx playwright test tests/login.spec.ts
```

Running Tests in Parallel

Playwright runs tests in parallel by default, using multiple worker processes. You can control parallelism at three levels:

Global Parallelism

```
// playwright.config.ts
export default defineConfig({
  fullyParallel: true, // All tests in all files run in parallel
  workers: 4, // Use 4 CPU cores
});
```

File-Level Parallelism

```
// tests/myTest.spec.ts
import { test } from '@playwright/test';

// Force serial execution in this file only
test.describe.configure({ mode: 'serial' });

test('first test', async ({ page }) => { /* ... */ });
```

```
test('second test', async ({ page }) => { /* ... */ });
```

Using Retries & Workers

Retries re-run a failed test automatically. **Workers** control how many tests execute simultaneously. Together they balance speed vs. stability.

```
export default defineConfig({  
  // Global retry setting  
  retries: 2,  
  
  // Worker configuration  
  workers: process.env.CI ? 2 : '50%', // 50% of CPUs locally  
});
```

```
// ■■ Per-test retry override ■■  
test('flaky network test', async ({ page }) => {  
  // ...  
}, { retries: 3 }); // override globally defined retries
```

■ *Note: Use `test.info().retry` to detect whether a test is being retried and take conditional action (e.g., clear cookies on retry).*

```
test('retry-aware test', async ({ page }) => {  
  if (test.info().retry > 0) {  
    // Clear cookies on retry attempts  
    await page.context().clearCookies();  
  }  
  await page.goto('/');  
  // rest of test...  
});
```

CHAPTER 03

Test Suite & Test Level Configurations

Screenshots in HTML Report • Suite Organisation

Structuring Test Suites

Playwright uses **test.describe()** to group related tests into a suite. Suites can be nested and can carry their own configuration, annotations, and hooks.

```
// tests/userManagement.spec.ts
import { test, expect } from '@playwright/test';

test.describe('User Management', () => {

  test.describe('Registration', () => {
    test('Valid registration', async ({ page }) => {
      await page.goto('/register');
      await page.fill('[name=email]', 'new@test.com');
      await page.fill('[name=password]', 'Pass@123');
      await page.click('button[type=submit]');
      await expect(page.locator('.success-msg')).toBeVisible();
    });

    test('Duplicate email error', async ({ page }) => {
      await page.goto('/register');
      await page.fill('[name=email]', 'existing@test.com');
      await page.click('button[type=submit]');
      await expect(page.locator('.error-msg')).toContainText('already exists');
    });
  });

  test.describe('Profile', () => {
    test('Update display name', async ({ page }) => {
      // ...
    });
  });
});
```

Test-Level Configuration Overrides

Individual tests can override global config using **test.use()** or inline options. This is useful when a specific test needs a different viewport, locale, or browser context.

```
import { test, expect } from '@playwright/test';

// ■■ Suite-level override ■■
test.describe('Mobile View Tests', () => {
  test.use({ viewport: { width: 375, height: 667 } });

  test('Mobile nav is visible', async ({ page }) => {
    await page.goto('/');
    await expect(page.locator('.mobile-nav')).toBeVisible();
  });
});

// ■■ Test-level override ■■
test('French locale checkout', async ({ page }) => {
  // Inline option override
  }, {
    use: { locale: 'fr-FR', timezoneId: 'Europe/Paris' },
    timeout: 60_000,
    retries: 1,
  });

// ■■ Annotations ■■
test('Known bug - skip for now', async ({ page }) => {
  test.skip(true, 'BUG-1234: Payment gateway down');

  // ...
});

test('Slow integration test', async ({ page }) => {
  test.slow(); // triples the timeout automatically
  // ...
});
```

Attaching Screenshots to HTML Report

Playwright's HTML reporter can embed screenshots directly in the test report. Screenshots can be captured automatically on failure or manually at any point using **testInfo.attach()**.

Automatic Screenshots on Failure

```
// playwright.config.ts
export default defineConfig({
  use: {
    // Options: 'off' | 'on' | 'only-on-failure'
    screenshot: 'only-on-failure',
  },
  reporter: [['html', { open: 'never' }]],
});
```

Manual Screenshot Attachment

```
import { test, expect } from '@playwright/test';

test('Cart checkout with screenshot evidence', async ({ page }, testInfo) => {
  await page.goto('/shop');
  await page.click('.add-to-cart');

  // Attach a screenshot at a key step
  const screenshotBytes = await page.screenshot();
  await testInfo.attach('cart-after-add', {
    body: screenshotBytes,
    contentType: 'image/png',
  });

  await page.click('.checkout-btn');
  await expect(page.locator('.order-confirmation')).toBeVisible();

  // Final screenshot
  const finalShot = await page.screenshot({ fullPage: true });
  await testInfo.attach('order-confirmation', {
    body: finalShot,
    contentType: 'image/png',
  });
});
```

■ Tip: Run `npx playwright show-report` after tests to open the interactive HTML report. Screenshots and traces are embedded inline in each test row.

Attaching Other Artifacts

```
// Attach a text log
await testInfo.attach('api-response.json', {
  body: JSON.stringify(apiResponse, null, 2),
  contentType: 'application/json',
});
```

```
// Attach a downloaded file
await testInfo.attach('invoice.pdf', {
  path: './downloads/invoice.pdf',
  contentType: 'application/pdf',
});
```

CHAPTER 04

Tracing & Debugging

Playwright Trace Viewer • Inspector • Debug Flags

What is Tracing?

Playwright **Trace** is a ZIP file that records a full timeline of your test: every action, network request, DOM snapshot, screenshot, and console log. It is the best tool for post-mortem debugging of CI failures.

Enabling Traces in Config

```
// playwright.config.ts
export default defineConfig({
  use: {
    // 'off' - never record
    // 'on' - always record
    // 'retain-on-failure' - keep only for failed tests
    // 'on-first-retry' - record on the first retry
    trace: 'on-first-retry',
  },
});
```

Manual Trace Control

```
test('Manual tracing', async ({ page, context }) => {
  // Start tracing
  await context.tracing.start({ screenshots: true, snapshots: true });

  await page.goto('/checkout');
  await page.click('.pay-btn');

  // Stop and save trace
  await context.tracing.stop({ path: 'traces/checkout.zip' });
});
```

Viewing Traces

```
# Open trace viewer for a specific trace file
npx playwright show-trace traces/checkout.zip

# Or open from the HTML report (traces are embedded automatically)
npx playwright show-report
```

Playwright Inspector

The Playwright Inspector is a GUI tool for stepping through tests, inspecting elements, and generating locators interactively.

```
# Open Inspector - pauses before each action

PWDEBUG=1 npx playwright test tests/login.spec.ts

# Or add pause() in your test code

test('Debug pause example', async ({ page }) => {

  await page.goto('/login');

  await page.pause(); // <-- Inspector opens here

  await page.fill('#email', 'user@test.com');

});
```

Verbose Console Debugging

```
// Listen to console messages from the browser

page.on('console', msg => {

  console.log(`[Browser ${msg.type()}] ${msg.text()}`);

});

// Listen for page errors

page.on('pageerror', err => {

  console.error(`Page error: ${err.message}`);

});

// Listen for network requests

page.on('request', req => console.log(`>> ${req.method()} ${req.url()}`));

page.on('response', resp => console.log(`<< ${resp.status()} ${resp.url()}`));
```

Debugging Flags Cheat Sheet

Flag / Option	What It Does
PWDEBUG=1	Open Inspector before each action
--headed	Run tests in headed (visible) mode
--timeout=0	Disable test timeout (infinite wait)
--grep "test name"	Run only tests matching a pattern
--debug	Start in debug mode with step-through
page.pause()	Insert breakpoint in test code
slowMo: 500	Slow down actions by 500ms in config

■ Tip: When debugging on CI, set trace: 'on' and download the trace artifact. You can replay the trace locally with full visual context even without a browser.

CHAPTER 05

Playwright Hooks

beforeAll • afterAll • beforeEach • afterEach & Practice Scripts

Overview of Hooks

Hooks are lifecycle callbacks that run automatically before or after tests. They are used to set up preconditions (login state, DB seeding) and clean up afterwards (logout, delete test data). Playwright provides four hooks:

Hook	Scope	Runs
beforeAll	describe block	Once before all tests in the block
afterAll	describe block	Once after all tests in the block
beforeEach	describe block	Before every individual test
afterEach	describe block	After every individual test

beforeAll & afterAll

beforeAll runs once before all tests in a describe block. Use it for expensive setup: creating a shared browser context, seeding a database, or generating auth tokens. **afterAll** cleans up those resources.

```

import { test, expect, Browser, BrowserContext, Page } from '@playwright/test';

test.describe('Admin Panel Tests', () => {

  let browser: Browser;

  let context: BrowserContext;

  let adminPage: Page;

  // Runs ONCE before all tests in this describe
  test.beforeAll(async ({ browser: b }) => {

    browser = b;

    context = await browser.newContext({
      storageState: './auth/admin.json', // pre-logged-in state
    });

    adminPage = await context.newPage();
    await adminPage.goto('/admin');
    console.log('Admin session created');
  });

  // Runs ONCE after all tests complete
  test.afterAll(async () => {

    await adminPage.close();
    await context.close();
    console.log('Admin session closed');
  });

  test('Can view user list', async () => {

    await adminPage.click('nav >> text=Users');

    await expect(adminPage.locator('.user-table')).toBeVisible();
  });

  test('Can delete a test user', async () => {

    await adminPage.click('.delete-btn >> nth=0');

    await expect(adminPage.locator('.toast')).toContainText('Deleted');
  });
});

```

beforeEach & afterEach

beforeEach runs before every single test. Use it to navigate to a base URL, reset state, or set up fixtures. **afterEach** runs after every test — ideal for cleanup, logging, or taking screenshots on failure.

```
import { test, expect } from '@playwright/test';

import { loginUser } from '../utils/helpers';

test.describe('Shopping Cart', () => {

  // Runs before EVERY test - login & navigate
  test.beforeEach(async ({ page }) => {

    await loginUser(page, 'customer@test.com', 'pass123');

    await page.goto('/shop');

    console.log(`Starting test: ${test.info().title}`);

  });

  // Runs after EVERY test - cleanup
  test.afterEach(async ({ page }, testInfo) => {

    if (testInfo.status !== testInfo.expectedStatus) {

      // Capture failure screenshot
      const screenshot = await page.screenshot();

      await testInfo.attach('failure-screenshot', {

        body: screenshot, contentType: 'image/png',

      });

    }

    // Clear cart after each test
    await page.evaluate(() => localStorage.removeItem('cart'));

  });

  test('Add item to cart', async ({ page }) => {

    await page.click('.product:first-child .add-to-cart');

    await expect(page.locator('.cart-count')).toHaveText('1');

  });

  test('Remove item from cart', async ({ page }) => {

    await page.click('.product:first-child .add-to-cart');

    await page.click('.cart-icon');

    await page.click('.remove-item');

    await expect(page.locator('.empty-cart-msg')).toBeVisible();

  });

});
```

Complete Practice Script

The following example combines all four hooks with assertions, utility imports, and screenshot attachment in one complete, production-ready spec:

```
// tests/e2e/orderFlow.spec.ts
import { test, expect } from '@playwright/test';

import { loginUser, generateUser } from '../../utils/dataUtils';

test.describe('Order Flow E2E', () => {

  const user = generateUser({ name: 'Hook Tester' });

  test.beforeAll(async ({ request }) => {

    // Seed test user via API before any test runs
    await request.post('/api/users', { data: user });

    console.log(`[beforeAll] Created user: ${user.email}`);

  });

  test.afterAll(async ({ request }) => {

    // Delete test user after all tests are done
    await request.delete(`/api/users/${user.email}`);

    console.log(`[afterAll] Cleaned up user`);

  });

  test.beforeEach(async ({ page }) => {

    await loginUser(page, user.email, 'TestPass@123');

  });

  test.afterEach(async ({ page }, testInfo) => {

    console.log(`[afterEach] ${testInfo.title} -> ${testInfo.status}`);

    if (testInfo.status === 'failed') {

      await testInfo.attach('screenshot', {

        body: await page.screenshot({ fullPage: true }),

        contentType: 'image/png',

      });

    }

  });

  test('Browse and add product', async ({ page }) => {

    await page.goto('/products');

    await page.click('.product-card:first-child');

    await page.click('#add-to-cart');

    await expect(page.locator('.cart-badge')).toHaveText('1');

  });

  test('Complete checkout', async ({ page }) => {
```

```
await page.goto('/cart');  
await page.click('#checkout-btn');  
await page.fill('[name=address]', '123 Test St');  
await page.click('#place-order');  
await expect(page.locator('.order-success')).toBeVisible();  
});  
});
```

■ Tip: Hooks inherit the same fixture types (page, context, request, browser) as tests. You can use any built-in Playwright fixture inside a hook.

CHAPTER 06

Framework Explanation

Architecture • Page Object Model • Project Structure

Playwright Architecture Overview

Playwright is an end-to-end testing framework developed by Microsoft. It uses the Chrome DevTools Protocol (CDP) for Chromium, and equivalent protocols for Firefox and WebKit. Each test gets an isolated browser context (like an incognito window), ensuring no state leaks between tests.

Layer	Component	Responsibility
Test Runner	Playwright Test	Discovers, schedules, and runs specs
Browser Control	Browser / BrowserContext	Manages browser sessions
Page	Page / Frame	Represents a tab; interacts with DOM
Network	Request / Response	Intercept, mock, and spy on HTTP
Assertion	expect()	Auto-retrying, async-aware assertions
Reporting	Reporters (HTML, JSON)	Outputs test results and artifacts

Page Object Model (POM)

The Page Object Model is a design pattern where each page (or component) of your application has a corresponding TypeScript class. This class encapsulates all locators and actions for that page, keeping tests clean and maintainable.

Page Object Class

```
// pages/LoginPage.ts
import { Page, Locator, expect } from '@playwright/test';

export class LoginPage {

  readonly page: Page;

  readonly emailInput: Locator;

  readonly passwordInput: Locator;

  readonly submitBtn: Locator;

  readonly errorMsg: Locator;

  constructor(page: Page) {

    this.page = page;

    this.emailInput = page.locator('#email');

    this.passwordInput = page.locator('#password');

    this.submitBtn = page.locator('button[type=submit]');

    this.errorMsg = page.locator('.error-message');

  }

  async goto() {

    await this.page.goto('/login');

  }

  async login(email: string, password: string) {

    await this.emailInput.fill(email);

    await this.passwordInput.fill(password);

    await this.submitBtn.click();

  }

  async expectError(message: string) {

    await expect(this.errorMsg).toContainText(message);

  }

}
```

Using POM in Tests


```
// tests/login.spec.ts
import { test, expect } from '@playwright/test';

import { LoginPage } from '../pages/LoginPage';

test('Valid login redirects to dashboard', async ({ page }) => {
  const loginPage = new LoginPage(page);
  await loginPage.goto();
  await loginPage.login('admin@test.com', 'Admin@123');
  await expect(page).toHaveURL('/dashboard');
});

test('Invalid credentials show error', async ({ page }) => {
  const loginPage = new LoginPage(page);
  await loginPage.goto();
  await loginPage.login('wrong@test.com', 'badpass');
  await loginPage.expectError('Invalid credentials');
});
```

Custom Fixtures

Playwright fixtures extend the test object to automatically provide page objects, pre-authenticated contexts, and shared state to every test that needs them.

```
// fixtures/index.ts
import { test as base } from '@playwright/test';

import { LoginPage } from '../pages/LoginPage';
import { DashboardPage } from '../pages/DashboardPage';

type Fixtures = {
  loginPage: LoginPage;
  dashboardPage: DashboardPage;
};

export const test = base.extend<Fixtures>({
  loginPage: async ({ page }, use) => {
    const lp = new LoginPage(page);
    await lp.goto();
    await use(lp);
  },
  dashboardPage: async ({ page }, use) => {
    await use(new DashboardPage(page));
  },
});

export { expect } from '@playwright/test';

// ■■ In your tests ■■
// import { test, expect } from '../fixtures';
// test('...', async ({ loginPage }) => { ... });
```

Recommended Project Structure

```

my-playwright-project/
├── playwright.config.ts # Global config
├── package.json
├── tsconfig.json
├── auth/
│   └── admin.json # Saved auth state
├── fixtures/
│   └── index.ts # Custom fixtures
├── pages/ # Page Object classes
│   ├── LoginPage.ts
│   ├── DashboardPage.ts
│   └── CheckoutPage.ts
├── utils/ # Utility functions
│   ├── index.ts # Barrel export
│   ├── helpers.ts
│   ├── dataUtils.ts
│   └── ApiClient.ts
├── tests/
│   ├── auth/
│   │   ├── login.spec.ts
│   │   └── e2e/
│   │       └── orderFlow.spec.ts
│   └── api/
│       ├── userApi.spec.ts
│       └── test-data/ # JSON test data files
│           └── users.json
└── playwright-report/ # HTML report output

```

■ Tip: Store auth/admin.json generated by npx playwright codegen or a setup project. It allows tests to skip login and run much faster.

Key Playwright CLI Commands

Command	Purpose
npx playwright test	Run all tests
npx playwright test --headed	Run in headed mode
npx playwright test --ui	Open interactive UI mode
npx playwright codegen <url>	Record test code automatically
npx playwright show-report	Open HTML report

<code>npx playwright install</code>	Install browser binaries
<code>npx playwright test --debug</code>	Run with Inspector

Summary

- Utilities and modular imports keep your test code DRY and maintainable.
- `playwright.config.ts` is the single source of truth for browser, parallelism, retry, and reporter settings.
- Cross-browser testing is achieved through the `projects` array — no extra code needed.
- Hooks (`beforeAll`, `afterAll`, `beforeEach`, `afterEach`) manage test lifecycle cleanly.
- Tracing and the HTML reporter give you full visibility into failures on CI.
- The Page Object Model combined with custom fixtures creates a scalable framework architecture.

End of Material • Happy Testing with Playwright!